

Python Code

```
1 class Graph:
2     def __init__(self, adjacency_list):
3         self.adj_list = adjacency_list
4
5 class Vertex:
6     def __init__(self, value=None):
7         self.value = None
8
9 class Edge:
10     def __init__(self, distance=None, vertex=None):
11         self.distance = distance
12         self.vertex = vertex
13
14 class Queue:
15     def __init__(self):
16         # self.queues = [(2, "A"), (4, "B"), (5, "C")]
17         self.queues = []
18     def enqueue(self, dist, vertex):
19         self.queues.append((dist, vertex))
20         self.queues.sort(key=lambda x: x[0])
21     def dequeue(self):
22         result = self.queues.pop(0)
23         return result[0], result[1]
24     def length_queue(self):
25         return len(self.queues)
26
27     def printing(self):
28         return self.queues
29 # q = Queue()
30 # q.enqueue(1, "Z")
31 # print(q.printing())
32 # print(q.dequeue())
33 # print(q.printing())
34
35 def dijkstra_alog(graph, start):
36     prev = { v:None for v in graph.adj_list.keys()}
37     visited = { v:False for v in graph.adj_list.keys()}
38     dist = { v:float("inf") for v in graph.adj_list.keys()}
39     dist[start] = 0
40     Queues = Queue()
41     Queues.enqueue(0, start)
42     while Queues.length_queue()>0:
43         removed_dist, remove = Queues.dequeue()
44         visited[remove] = True
45         for edge in graph.adj_list[remove]:
46             if visited[edge.vertex] == True:
47                 continue
48             new_distance = removed_dist+edge.distance
49             if new_distance<dist[edge.vertex]:
50                 dist[edge.vertex] = new_distance
51                 prev[edge.vertex] = remove
52                 Queues.enqueue(new_distance, edge.vertex)
```

```

53     return dist
54
55     vertices = [ Vertex("A"),
56                 Vertex("B"),
57                 Vertex("C"),
58                 Vertex("D"),
59                 Vertex("E"),
60                 Vertex("F"),
61                 Vertex("G"),
62                 Vertex("H")]
63     adj_list = {
64         "A": [Edge(2, "B"), Edge(4, "C")],
65         "B": [Edge(1, "C"), Edge(7, "D")],
66         "C": [Edge(3, "E")],
67         "D": [ Edge(1, "F")],
68         "E": [Edge(2, "D"), Edge(3, 'F')],
69         "F": []
70     }
71
72     my_graph = Graph(adj_list)
73
74     g =dijkstra_alog(my_graph, start="A")
75     print(g)
76
77
78     # queues = [(2,"A"),(4,"B"),(5,"C")]
79     # graph = {
80     #         "A" : ["B","C","D"],
81     #         "B" : ["A","C","E"],
82     #         "c" : ["A","B","E","F"],
83     #         "D" : ["A","F"],
84     #         "E" : ["B","C","F"],
85     #         "F" : ["C","D","E"]
86     #     }
87
88
89     # testing the algorithm
90     # vertices = [ Vertex("A"),
91                 Vertex("B"),
92                 Vertex("C"),
93                 Vertex("D"),
94                 Vertex("E"),
95                 Vertex("F"),
96                 Vertex("G"),
97                 Vertex("H")]
98     # A, B, C, D, E, F, G, H = vertices
99     # adj_list = {
100     #     "A": [Edge(1.8, "B"), Edge(1.5, "C"), Edge(1.4, "D")],
101     #     "B": [Edge(1.8, "A"), Edge(1.6, "E")],
102     #     "C": [Edge(1.5, "A"), Edge(1.8, "E"), Edge(2.1, "F")],
103     #     "D": [Edge(1.4, "A"), Edge(2.7, "F"), Edge(2.4, "G")],
104     #     "E": [Edge(1.6, "B"), Edge(1.8, "C"), Edge(1.4, 'F'), Edge(1.6,
105     #         "H")],
106     #     "F": [Edge(2.1, "C"), Edge(2.7, "D"), Edge(1.4, "E"), Edge(1.3, "G"),
107     #         Edge(1.2, "H")],

```

```

106 # "G": [Edge(2.4, "D"), Edge(1.3, "F"), Edge(1.5, "H")],
107 # "H": [Edge(1.6, "E"), Edge(1.2, "F"), Edge(1.5, "G")],
108 # }
109 #0/1 Knapsack (Classic Beginner DP)
110 # _____ knapsack problems _____

```

```

111 P = [0,1,2,5,6]
112 Wt = [0,2,3,4,5]
113 m = 8
114 n = 4
115 K = [[ 0 for j in range(m+1)]for i in range(n+1)]
116
117 for i in range(n+1):
118     for w in range(m+1):
119         if i == w == 0:
120             K[i][w] = 0
121         elif Wt[i]<=w:
122             K[i][w] = max(P[i]+K[i-1][w-Wt[i]],K[i-1][w])
123         else:
124             K[i][w] = K[i-1][w]
125
126
127
128 for i in K:
129     print(i)
130
131
132
133 #####
#####
134 # Topological sort
135 from collections import deque
136
137 def topological_sort_kahn(graph):
138     """
139     Performs a topological sort on a Directed Acyclic Graph (DAG)
140     using Kahn's Algorithm (In-degree method), based on the provided steps.
141
142     Args:
143         graph (dict): A dictionary representing the graph where keys are ve
rtices
144                        and values are lists of adjacent vertices (neighbor
s).
145                        Example: {0: [1, 2], 1: [3], 2: [3], 3: []}
146
147     Returns:
148         list: A topologically sorted list of vertices, or an empty list if
a cycle is detected.
149     """
150
151     # 1. Store each vertex's In-Degree in an array D
152     # Initialize in-degree for all vertices to 0.
153     in_degree = {u: 0 for u in graph}
154
155     # Calculate actual in-degrees

```

```

156     for u in graph:
157         for v in graph[u]:
158             in_degree[v] += 1
159
160     # --- Initialization ---
161
162     # 2. Initialize queue with all "in-degree=0" vertices
163     queue = deque()
164     for u in graph:
165         if in_degree[u] == 0:
166             queue.append(u)
167
168     topological_order = []
169
170     # 3. While there are vertices remaining in the queue
171     while queue:
172         # (a) Dequeue and output a vertex
173         u = queue.popleft()
174         topological_order.append(u)
175
176         # Iterate over neighbors v
177         for v in graph[u]:
178             # (b) Reduce In-Degree of all vertices adjacent to it by 1
179             in_degree[v] -= 1
180
181             # (c) Enqueue any of these vertices whose In-Degree became zero
182             if in_degree[v] == 0:
183                 queue.append(v)
184
185     # --- Cycle Detection ---
186
187     # 4. If all vertices are output then success, otherwise there is a cycl
e.
188     if len(topological_order) == len(graph):
189         return topological_order
190     else:
191         # A cycle exists if the number of sorted vertices is less than the
total number of vertices.
192         # Note: Depending on requirements, you might return None, an error
message, or the partial sort.
193         # Returning an empty list here to signify failure/cycle detection.
194         print("Error: The graph contains a cycle. Topological sort is not p
ossible.")
195         return []
196
197     # --- Example Usage ---
198
199     # Example DAG (Directed Acyclic Graph):
200     # A -> C, B -> C, B -> D, C -> E, D -> E
201     # Vertices: 5 (A, B, C, D, E)
202     # In-degrees: A:0, B:0, C:2, D:1, E:2
203     graph = {
204         'A': ['C'],
205         'B': ['C', 'D'],
206         'C': ['E'],

```

```

207     'D': ['E'],
208     'E': []
209 }
210
211 sorted_list = topological_sort_kahn(graph)
212 print(f"\nTopological Sort Order: {sorted_list}")
213
214 # Example Graph with a Cycle (for step 4 verification)
215 # # A -> B -> C -> A
216 # cycle_graph = {
217 #     1: [2],
218 #     2: [3],
219 #     3: [1]
220 # }
221
222 # sorted_list_cycle = topological_sort_kahn(cycle_graph)
223 # print(f"\nTopological Sort Order (Cycle Test): {sorted_list_cycle}")
224
225 #####
#####
226 # _____Bubble Sort_____
#
227 l = [9,8,7,6,5,4,3,2,1]
228 def Bubble_Sort(l):
229     n = len(l)
230     for i in range(n):
231         for j in range(n-i-1):
232             if l[j]>l[j+1]:
233                 l[j], l[j+1] = l[j+1], l[j]
234     return l
235 print("Sorting")
236 print(Bubble_Sort(l))
237
238 #####
239 # _____Insertion Sort_____
#
240 l_insertion = [9, 8, 7, 6, 5, 4, 3, 2, 1]
241
242 def Insertion_Sort(l):
243     # Traverse from the second element (index 1) up to the end
244     for i in range(1, len(l)):
245
246         key = l[i] # The element to be inserted
247         j = i - 1 # Start comparing with the element just before 'key'
248
249         # Move elements of the sorted part (l[0..i-1]) that are greater tha
n key
250         # to one position ahead of their current position
251         while j >= 0 and key < l[j]:
252             l[j + 1] = l[j]
253             j -= 1
254
255         # Place the key in its correct position
256         l[j + 1] = key
257

```

```

258     return l
259
260 print("\nSorting (Insertion Sort)")
261 print(Insertion_Sort(l_insertion))
262 #####
263 # _____ Merge Sort _____
    #
264 l_merge = [9, 8, 7, 6, 5, 4, 3, 2, 1]
265
266 def Merge_Sort(l):
267     if len(l) > 1:
268
269         mid = len(l) // 2 # Find the middle point
270         L = l[:mid]       # Divide the elements into two halves
271         R = l[mid:]
272
273         # Recursively sort the first and second halves
274         Merge_Sort(L)
275         Merge_Sort(R)
276
277         i = j = k = 0
278
279         # Merge the two sorted halves (L and R) back into l[]
280         while i < len(L) and j < len(R):
281             if L[i] < R[j]:
282                 l[k] = L[i]
283                 i += 1
284             else:
285                 l[k] = R[j]
286                 j += 1
287             k += 1
288
289         # Copy the remaining elements of L[], if any
290         while i < len(L):
291             l[k] = L[i]
292             i += 1
293             k += 1
294
295         # Copy the remaining elements of R[], if any
296         while j < len(R):
297             l[k] = R[j]
298             j += 1
299             k += 1
300
301     return l
302
303 print("\nSorting (Merge Sort)")
304 print(Merge_Sort(l_merge))
305 #####
306 # _____ Quick Sort _____
    #
307 l_quick = [9, 8, 7, 6, 5, 4, 3, 2, 1]
308
309 def Quick_Sort(l):
310     if len(l) <= 1:

```

```

311         return l
312     else:
313         # Choose the first element as the pivot for simplicity
314         pivot = l[0]
315
316         # Partition the list into three parts: less than pivot, equal to pi
vot, greater than pivot
317         less = [x for x in l[1:] if x <= pivot]
318         greater = [x for x in l[1:] if x > pivot]
319
320         # Recursively sort the partitions and combine the result
321         return Quick_Sort(less) + [pivot] + Quick_Sort(greater)
322
323     print("\nSorting (Quick Sort)")
324     print(Quick_Sort(l_quick))
325     #####
###
326
327     # _____ Bucket sort _____ #
328     def bucketsort(l):
329
330         bucket = [[] for i in range(10)]
331
332         for i in l:
333             bucket[i].append(i)
334
335         out = []
336         for i in bucket:
337             for j in i:
338                 out.append(j)
339
340         return out
341     print("_____ bucketsort _____")
342     print(bucketsort(l))
343
344     #####
#####
345     # _____ radix_sort _____ #
346     def counting_sort(arr, place):
347         size = len(arr)
348         output = [0] * size      # temporary array
349         count = [0] * 10         # digits 0-9
350
351         # Count the occurrences of each digit
352         for num in arr:
353             digit = (num // place) % 10
354             count[digit] += 1
355
356         # Convert count[i] to actual positions
357         for i in range(1, 10):
358             count[i] += count[i - 1]
359
360         # Build the output array (stable)
361         for i in range(size - 1, -1, -1):
362             digit = (arr[i] // place) % 10

```

```
363         output[count[digit] - 1] = arr[i]
364         count[digit] -= 1
365
366     # Copy output back into original array
367     for i in range(size):
368         arr[i] = output[i]
369
370
371 def radix_sort(arr):
372     # Find the maximum number to know number of digits
373     max_num = max(arr)
374
375     place = 1
376     # Apply counting sort for each digit (units, tens, hundreds...)
377     while max_num // place > 0:
378         counting_sort(arr, place)
379         place *= 10
380
381
382 # Example usage:
383 arr = [170, 45, 75, 90, 802, 24, 2, 66]
384 radix_sort(arr)
385 print("Sorted array:", arr)
386
```